**HOCHSCHULE**
**HANNOVER**
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
*Fakultät IV*
*Wirtschaft und*
*Informatik*

# AISoftDev Project Work and Examination Task Description

AI in Software Development WS 2026/27

Prof. Dr. Andreas Hinderks

September 19, 2026



# Contents

**1 General Information on the Project Work** **3**
1.1 Tools and Costs . . . . . 3
1.2 Use of AI Tools . . . . . 4

**2 Task / Project Work Description** **5**
2.1 Project Pitch . . . . . 5
2.2 Milestone 1: The Harness Runs . . . . . 6
2.3 Milestone 2: The Verification Loop . . . . . 6
2.4 Milestone 3: The Evaluation Set . . . . . 7
2.5 Milestone 4: The Security Review . . . . . 7
2.6 Presentation Information . . . . . 8
2.7 Final Submission . . . . . 8
2.8 Assessment . . . . . 8

# 1 General Information on the Project Work

This project work runs for the whole semester and is your examination performance. Unlike a conceptual project, this one requires programming: your team takes an existing coding agent (standard: OpenCode) and engineers a *harness* for it, so that the agent reliably builds software in a domain of your choice. You do not implement the agent loop yourself, but you must be able to explain it. You then use the agent with your harness to build a piece of software in that domain. Both results, the harness and the software built with it, are assessed.

Your harness is a repository of its own. It contains context files (AGENTS.md, rules, specification), domain tools you implement yourself, a permission and sandbox concept, a verification loop (tests, type check, build or simulation as a signal the agent reacts to), hooks or commands, and an evaluation set with a runner. Harness engineering means programming, not only writing Markdown: every milestone contains a part you have to implement.

The module follows a competency model with eight competencies (K1 to K8), which you will receive in the first lecture. Every milestone is assessed against these competencies, not against the number of features you ship. A small harness that verifiably works, is evaluated with data, and is documented honestly scores higher than a large one that nobody can explain.

## 1.1 Tools and Costs

The standard tool of the module is OpenCode (open source, MIT licence). The reference version for lectures and exercises is OpenCode 1.18.31. Install exactly this version (`npm install -g opencode-ai@1.18.31`) and disable automatic updates; I will announce in the exercise if the pinned version changes. You are free to switch to OpenCode 2; note that its plugin API differs from version 1. You may also use another coding agent as the base of your harness, provided it offers context files, custom tools, permissions and a headless mode. Claude Code, Codex CLI and similar tools are shown in the lectures for comparison.

Model access is provided in two ways. First, the MDI lab offers two Mac Studios and four PCs on which open-weights models (Qwen) run locally; every team can use them for setting up, testing and running its harness. Second, the university is currently checking whether API usage of commercial models can be billed centrally; if so, each team receives a budget, and I will announce the amount and the rules in the exercise. You may also use models and providers you have access to yourself, but you are never required to. Assessment is tool-neutral: a harness that runs with a local open-weights model is not graded lower than one that runs with a frontier model, and the choice of the base agent does not affect the grade.

The programming language follows from your base agent. In OpenCode, tool definitions are written in TypeScript or JavaScript, but the logic behind a tool may be a script in any language. The reference code shown in the lectures is TypeScript. In lecture 5 you receive `mini-agent`, a coding agent in about 100 lines. You read it, run it and modify it once in the exercise in order to understand the agent loop, but you do not build your project on it.

## 1.2 Use of AI Tools

Using AI tools is the subject of this module, so of course you may use them. Two rules apply. First, you must be able to explain every part of your harness and your software in the presentation and answer questions about it. Second, your documentation must state which parts were generated by an agent and how you reviewed them.

# 2 Task / Project Work Description

Form teams of three (two only if the total number of students requires it). Each team chooses a *domain* for its harness, for example database-driven web applications, browser games, command-line tools, or data pipelines, and a concrete piece of *target software* in that domain that the agent will build with your harness. Each team works in two repositories: the harness repository and the target software repository. Register your team and domain in Moodle by **October 4, 2026 at 23:59**, before the first exercise. Registering the project does not replace the formal exam registration; both must be completed separately.

The project is structured into a pitch, four milestones (M1 to M4), the final submission, and the presentation. Milestones are presented in the exercise of the stated week. For M1 and M2, the files are uploaded to Moodle by the Sunday before the exercise, so that I can look at them in advance, and the result is demonstrated live. For M3 and M4, you discuss your draft in the exercise and upload the final version by the Sunday after, so that you can incorporate the feedback.

## 2.1 Project Pitch

In the exercise of the week of October 12, 2026, each team gives a five-minute pitch covering:

- the domain and the target software (what it does, for whom),
- the result of a first test run: give OpenCode a small task in your domain and observe what it does well and where it fails,
- a diagnosis of the observed failures based on the properties of language models discussed in lecture 2 (competency K1).

Upload your slides to Moodle by **October 11, 2026 at 23:59**. The pitch is not graded separately, but the diagnosis feeds into the documentation.

## 2.2 Milestone 1: The Harness Runs

The first milestone shows that your team has set up a coding agent for your domain and extended it for the first time. Deliverables:

- A harness repository containing the configuration of your base agent, at least one tool you implemented yourself (for example a tool that runs your test suite and returns the failures in a structured form), and a README that explains how to install the harness into a project and how to run it. State the base agent, its version and the model. Built-in tools of the base agent do not count.
- A model selection: a criteria matrix comparing at least three models or providers by quality, cost, latency, licence and data protection, with a justified choice (K2).
- A context file (AGENTS.md or equivalent) and a written specification of your target software that the agent will work from (K3).
- A two-minute live demonstration in the exercise: the agent, running with your harness, completes a small task in your target repository and calls your tool.

Upload to Moodle by **November 1, 2026 at 23:59**; demonstration in the exercise of the week of November 2, 2026. Competencies: K3, K4; the criteria matrix (K2) is assessed with the documentation.

## 2.3 Milestone 2: The Verification Loop

The second milestone makes the agent verify its own work in your domain. Deliverables:

- At least two domain-specific tools you implemented yourself (for example running migrations, running tests, running a headless simulation of a game loop). The tool from M1 counts if it is domain-specific.
- A verification signal the agent reacts to: tests, a type checker, a build, or a simulation. The log must show that the agent received a failing signal, changed its work in response, and ran the check again. Define an explicit stop criterion (for example a step limit, a retry limit or a time budget) and show that it takes effect.
- A documented and configured sandbox and permission concept: what the agent may do without asking, what requires confirmation, what is forbidden. Your base agent may not provide a sandbox of its own; state how you isolate the agent (for example a container, a restricted working directory, deny rules).

- A demonstration in which the agent, running with your harness, implements one feature of your target software until the verification signal passes, plus the log of that run.

Upload to Moodle by **November 15, 2026 at 23:59**; demonstration in the exercise of the week of November 16, 2026. Competencies: K4, K6.

## 2.4 Milestone 3: The Evaluation Set

The third milestone measures your harness. Deliverables:

- An evaluation set of at least eight tasks in your domain, each with an expected outcome and an automatic check.
- A runnable eval runner: one command that executes all tasks headlessly (in OpenCode: `opencode run`), applies the automatic checks and writes the results table.
- A results table for your harness: success rate, token consumption or cost, and runtime per task.
- At least one improvement of the harness motivated by the results, with numbers before and after.
- A short interpretation: what the numbers say, what they do not say, and what you would measure next.

Discussion of your draft in the exercise of the week of November 23, 2026; final upload to Moodle by **November 29, 2026 at 23:59**. Competency: K5.

## 2.5 Milestone 4: The Security Review

The fourth milestone tests your harness against an attacker. Deliverables:

- An indirect prompt injection test: hidden instructions in a file, issue, or web page your agent reads. Document what happened.
- At least one countermeasure implemented in your harness (for example a permission rule, a hook that inspects tool calls, or a tool wrapper) and a repeat of the test. An instruction in the context file alone does not count.
- A risk assessment of your harness along three relevant entries of the OWASP Top 10 for LLM Applications (2025), including how secrets (API keys, credentials) are handled.

- A note on the licences of your dependencies and of the generated code.

Discussion of your draft in the exercise of the week of December 7, 2026; final upload to Moodle by **December 13, 2026 at 23:59**. Competency: K7.

## 2.6 Presentation Information

Presentations take place on **December 14, 2026** and **January 4, 2027**, in the lecture and the exercise slot. The allocation of teams to dates is announced in the exercise of the week of November 23, 2026. A dress rehearsal is part of the exercise of the week of December 7, 2026.

Each presentation lasts twenty minutes, not significantly shorter and not longer, followed by five minutes of questions. It must contain a live demonstration of the agent, running with your harness, building or changing something in the target software. Cover the architecture of the harness (base agent, extension points used, your tools, the verification loop), the most important design decisions and their justification, the evaluation results, the security review, and a short demonstration of the target software. Be prepared to explain the agent loop of your base agent. All team members must participate equally and must be able to answer questions about any part of the project. Prepare a video recording of the demo as a fallback in case the live demonstration fails.

## 2.7 Final Submission

**All artifacts must be uploaded to Moodle by January 08, 2027 at 23:59**: the harness repository (as a Git tag or archive), the target software repository, the documentation, the presentation slides, and the demo video. This is the basis of the assessment.

The documentation is at most ten pages and covers: the domain and target software, the base agent with its version and the architecture of the harness, the model selection, the verification loop, the evaluation results, the security review, a statement of which parts were agent-generated and how they were reviewed, and a statement of each member's contribution.

## 2.8 Assessment

The grade is composed as follows:

| Part | Weight | Competencies |
|---|---|---|
| Harness (M1, M2) | 35 % | K3, K4 |
| Target software | 20 % | K6 |
| Evaluation set (M3) | 15 % | K5 |
| Security review (M4) | 10 % | K7 |
| Presentation and documentation | 20 % | K1, K2, K8 |

For every milestone you receive a rubric in advance that describes three levels for each competency involved: basic, consolidated, advanced. Individual grades can deviate from the team grade based on the roles in the presentation, the commit history, and a short individual written reflection after the presentation.

Across all parts, the following criteria apply:

- **Functionality:** The agent runs with your harness, your tools, the verification loop and the eval runner work, and the demonstration succeeds or the failure is analysed.
- **Traceability and Justification:** Design decisions are documented and justified; evaluation and security claims are backed by data from your own runs.
- **Depth and Accuracy:** The concepts from the lectures (agent loop, tool design, context engineering, verification, evaluation, threat model) are correctly applied.
- **Clarity:** Documentation and presentation are precise and concise, without filler.
- **Honesty:** Limitations, failed runs, and agent-generated parts are stated openly. A documented failure counts; a hidden one does not.